

Assignment 1: Differential Drive Robot Control

Artyom Tuzov

<https://github.com/Artyom35689/AutMobRob-Assignment1>

1 Overview

The repository contains three ROS 2 packages:

- `my_robot_description`: URDF/Xacro model of the robot.
- `my_robot_bringup`: Gazebo launch files and basic bringup tools.
- `my_robot_controllers`: control nodes (P, Pure Pursuit, Stanley, MPC) and an odom plotter node.

Each controller publishes `/cmd_vel` and listens to `/odom`. The plotter listens to `/odom` and saves a figure into `plots/` when the robot reaches the final waypoint and remains stopped for a short settling time.

2 Robot Model

We use the standard unicycle kinematic model:

$$\dot{x} = v \cos \phi, \tag{1}$$

$$\dot{y} = v \sin \phi, \tag{2}$$

$$\dot{\phi} = \omega, \tag{3}$$

where v is commanded linear velocity and ω is commanded angular velocity. All controllers compute (v, ω) from the robot pose estimated by odometry.

3 Controllers

3.1 P Waypoint Controller

At each control step, the controller targets the current waypoint (x_t, y_t) and computes:

- Distance error: $\rho = \sqrt{(x_t - x)^2 + (y_t - y)^2}$
- Heading error: $\alpha = \text{wrap}(\text{atan2}(y_t - y, x_t - x) - \phi)$

Control law:

$$v = \text{clamp}(k_\rho \rho, 0, v_{\max}), \tag{4}$$

$$\omega = \text{clamp}(k_\alpha \alpha, -\omega_{\max}, \omega_{\max}). \tag{5}$$

If $|\alpha|$ is large, v is reduced to avoid “driving sideways” and oscillations. Waypoints are considered reached when $\rho \leq \text{goal_tolerance}$.

3.2 Pure Pursuit

Pure Pursuit selects a lookahead target point at distance L_d along the path and uses the curvature command:

$$\alpha = \text{wrap}(\text{atan2}(y_L - y, x_L - x) - \phi), \quad (6)$$

$$\kappa = \frac{2 \sin(\alpha)}{L_d}, \quad (7)$$

$$\omega = v\kappa. \quad (8)$$

Here v is typically constant (`v_cmd`) and then clamped by limits. Lookahead acts as the main tuning knob: smaller L_d makes tracking tighter but more oscillatory; larger L_d makes motion smoother but can cut corners.

3.3 Stanley Controller

Stanley uses two terms:

- Heading error: $e_\psi = \text{wrap}(\psi_{\text{path}} - \phi)$
- Cross-track error (signed): e_{cte} from projecting the robot position onto the closest path segment

Stanley steering law (originally for car-like steering angle δ):

$$\delta = e_\psi + \arctan\left(\frac{k e_{\text{cte}}}{v + v_0}\right). \quad (9)$$

In this project we map it to differential drive angular velocity via:

$$\omega = \frac{v}{L} \tan(\delta), \quad (10)$$

where L is an effective wheelbase parameter. In practice, δ is clamped to keep $\tan(\delta)$ bounded and stable. A robust implementation requires selecting the *closest* segment globally (not only a small window), otherwise the controller may lock onto a wrong segment and drive away.

3.4 Minimal MPC (Brute Force)

The MPC implementation is intentionally minimal:

- Predict the future state for N steps using the kinematic model and constant (v, ω) .
- Enumerate a small grid of candidate (v, ω) pairs.
- Choose the pair with minimal cost.

Cost (to a target waypoint) is:

$$J = \sum_{k=1}^N w_{\text{track}} \|p_k - p_t\|^2 + w_{\text{terminal}} \|p_N - p_t\|^2 + w_u (v^2 + c\omega^2), \quad (11)$$

where $p_k = (x_k, y_k)$ and $p_t = (x_t, y_t)$. This MPC is not a convex optimizer; it is a simple exhaustive search that is easy to understand and works well for small grids.

4 Data Collection and Visualization

A dedicated node (`odom_plotter`) subscribes to `/odom` and stores:

- XY position over time,
- Heading $\phi(t)$ (unwrapped for continuity),
- Final goal distance and a “stopped” indicator using odometry twist thresholds.

Stop condition for saving plots:

- Distance to the final waypoint $\leq \text{goal_tolerance}$, and
- The robot is stopped (small v, ω) for at least `settle_time`.

The saved figure includes waypoint markers and tolerance circles to visualize the acceptance radius.

5 Results

Figure 1 shows example outputs saved by the plotter node.

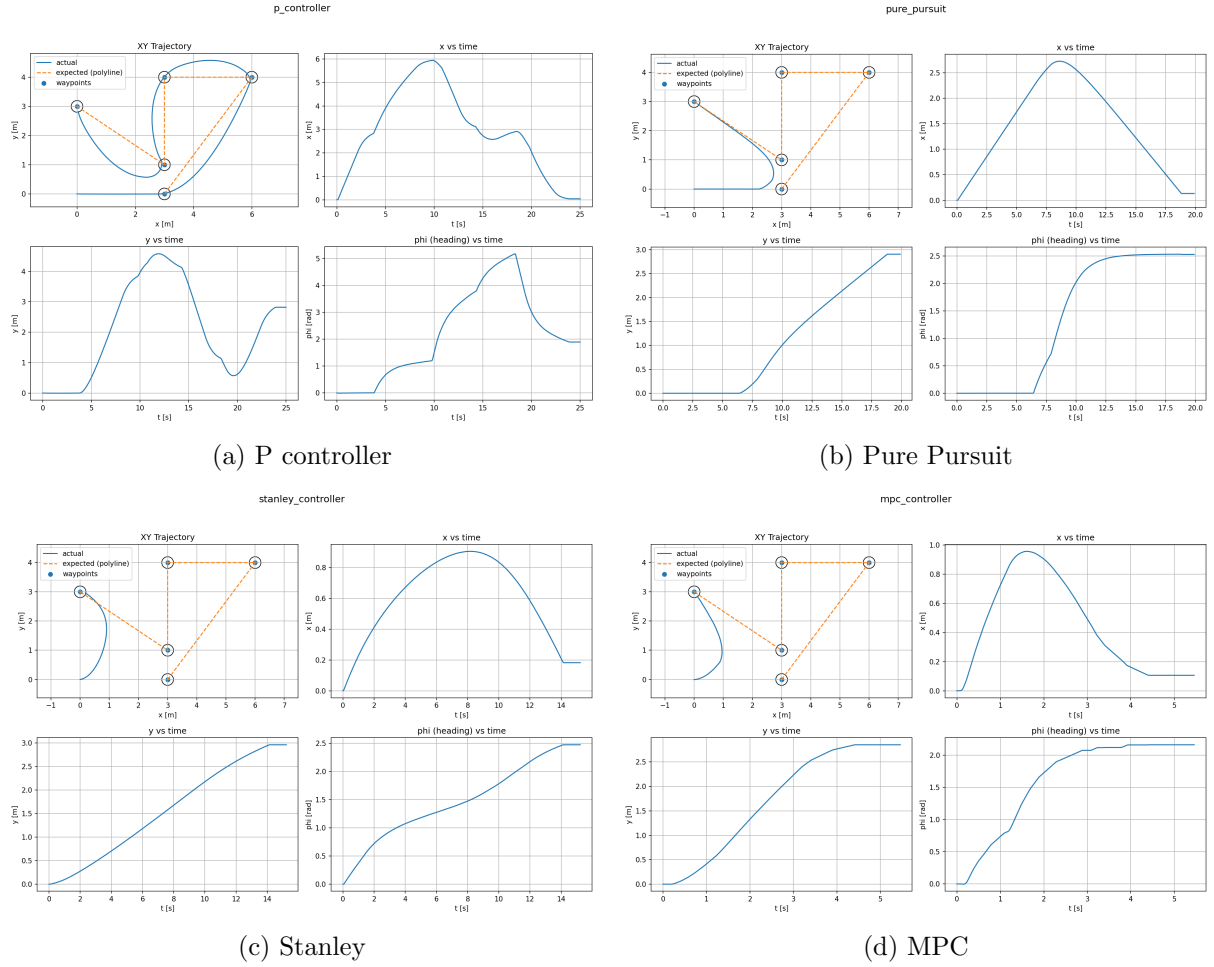


Figure 1: Recorded trajectory and time-series plots for each controller. Each image contains: XY trajectory, $x(t)$, $y(t)$, and $\phi(t)$.

6 Discussion

- **P controller** is straightforward and usually reaches waypoints but can oscillate, especially near corners, unless speed is reduced when heading error is large.
- **Pure Pursuit** is smooth and easy to tune; however, larger lookahead can cut corners, while smaller lookahead can produce oscillations.
- **Stanley** tracks path orientation and lateral error; correct signed cross-track error and robust closest-segment selection are critical for stable behavior.
- **MPC** (grid search) can produce visually smooth behavior by balancing tracking and control effort, but computational cost grows with grid size and horizon.

7 How to Run

Typical workflow:

1. Clone the repository.
2. Build the workspace (`colcon build`) and source `install/setup.bash`.
3. Launch a controller together with the plotter using provided launch files, e.g.:
 - `ros2 launch my_robot_controllers p_with_plots.launch.py`
 - `ros2 launch my_robot_controllers pure_pursuit_with_plots.launch.py`
 - `ros2 launch my_robot_controllers stanley_with_plots.launch.py`
 - `ros2 launch my_robot_controllers mpc_with_plots.launch.py`

After completion, plots are saved into `plots/`.

8 Conclusion

Four controllers (P, Pure Pursuit, Stanley, and a minimal MPC) were implemented and evaluated in simulation. A plotter node recorded trajectory and heading, producing consistent figures for comparison. The implementations demonstrate the trade-offs between simplicity (P), geometric tracking (Pure Pursuit), lateral-error correction (Stanley), and predictive cost-based control (MPC).